

Tarea N° 05

Funciones

Ejercicio 1: Media de Datos

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float datos[] = {23.5, 45.2, 33.8, 29.1};
5     float suma = 0;
6     for(int i = 0; i < 4; i++)
7         suma += datos[i];
8     printf("Media: %.2f\n", suma/4);
9     return 0;
10 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 float calcularMedia(float datos[], int n) {
4     float suma = 0;
5     for(int i = 0; i < n; i++) {
6         suma += datos[i];
7     }
8     return suma / n;
9 }
10
11 int main() {
12     float datos[] = {23.5, 45.2, 33.8, 29.1};
13     int n = 4;
14     float media = calcularMedia(datos, n);
15     printf("Media: %.2f\n", media);
16     return 0;
17 }
```

Salida

Media: 32.90

Ejercicio 2: Normalización de Valores

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float x = 75, min = 20, max = 100;
5     float normalizado = (x - min) / (max - min);
6     printf("Valor normalizado: %.3f\n", normalizado);
7     return 0;
8 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 float normalizar(float x, float min, float max) {
4     return (x - min) / (max - min);
5 }
6
7 int main() {
8     float x = 75, min = 20, max = 100;
9     float normalizado = normalizar(x, min, max);
10    printf("Valor normalizado: %.3f\n", normalizado);
11    return 0;
12 }
```

Salida

Valor normalizado: 0.688

Ejercicio 3: Clasificación de Temperatura

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float temp = 18.5;
5     if(temp < 10)
6         printf("Frio\n");
7     else if(temp < 25)
8         printf("Templado\n");
9     else
10        printf("Caliente\n");
11    return 0;
12 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void clasificarTemperatura(float temp) {
4     if(temp < 10)
5         printf("Frio\n");
6     else if(temp < 25)
7         printf("Templado\n");
8     else
9         printf("Caliente\n");
10 }
11
12 int main() {
13     float temp = 18.5;
14     clasificarTemperatura(temp);
15     return 0;
16 }
```

Salida

Templado

Ejercicio 4: Detección de Outliers (Z-Score)

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float valor = 95, media = 50, desv = 15;
5     float z_score = (valor - media) / desv;
6     if(z_score > 3 || z_score < -3)
7         printf("Outlier detectado\n");
8     return 0;
9 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 float calcularZScore(float valor, float media, float desv) {
4     return (valor - media) / desv;
5 }
6
7 int esOutlier(float z_score) {
8     return (z_score > 3 || z_score < -3);
9 }
10
11 int main() {
12     float valor = 95, media = 50, desv = 15;
13     float z_score = calcularZScore(valor, media, desv);
14
15     if(esOutlier(z_score))
16         printf("Outlier detectado\n");
17     else
18         printf("Valor normal\n");
19
20     return 0;
21 }
```

Salida

Valor normal

Ejercicio 5: Validación de Datos

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int edad = 150;
5     float peso = -5.5;
6     if(edad < 0 || edad > 120 || peso <= 0)
7         printf("Datos invalidos\n");
8     return 0;
9 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int validarDatos(int edad, float peso) {
4     if(edad < 0 || edad > 120 || peso <= 0)
5         return 0; // Datos invalidos
6     return 1;     // Datos validos
7 }
8
9 int main() {
10     int edad = 150;
11     float peso = -5.5;
12
13     if(!validarDatos(edad, peso))
14         printf("Datos invalidos\n");
15     else
16         printf("Datos validos\n");
17
18     return 0;
19 }
```

Salida

Datos invalidos

Ejercicio 6: Ventas Acumuladas

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int ventas[] = {100, 150, 200, 180, 220};
5     int acum = 0;
6     for(int i = 0; i < 5; i++) {
7         acum += ventas[i];
8         printf("Dia %d: Total=%d\n", i+1, acum);
9     }
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void mostrarVentasAcumuladas(int ventas[], int n) {
4     int acum = 0;
5     for(int i = 0; i < n; i++) {
6         acum += ventas[i];
7         printf("Dia %d: Total=%d\n", i+1, acum);
8     }
9 }
10
11 int main() {
12     int ventas[] = {100, 150, 200, 180, 220};
13     mostrarVentasAcumuladas(ventas, 5);
14     return 0;
15 }
```

Salida

```
Dia 1: Total=100
Dia 2: Total=250
Dia 3: Total=450
Dia 4: Total=630
Dia 5: Total=850
```

Ejercicio 7: Matriz de Correlación

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float corr[3][3] = {{1.0, 0.8, 0.6},
5                          {0.8, 1.0, 0.5},
6                          {0.6, 0.5, 1.0}};
7     for(int i = 0; i < 3; i++)
8         for(int j = 0; j < 3; j++)
9             printf("r[%d][%d]=%.1f ", i, j, corr[i][j]);
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void imprimirMatrizCorrelacion(float corr[3][3], int n) {
4     for(int i = 0; i < n; i++) {
5         for(int j = 0; j < n; j++) {
6             printf("r[%d][%d]=%.1f ", i, j, corr[i][j]);
7         }
8     }
9 }
10
11 int main() {
12     float corr[3][3] = {{1.0, 0.8, 0.6},
13                          {0.8, 1.0, 0.5},
14                          {0.6, 0.5, 1.0}};
15     imprimirMatrizCorrelacion(corr, 3);
16     return 0;
17 }
```

Salida

r[0][0]=1.0 r[0][1]=0.8 r[0][2]=0.6 r[1][0]=0.8 r[1][1]=1.0
r[1][2]=0.5 r[2][0]=0.6 r[2][1]=0.5 r[2][2]=1.0

Ejercicio 8: Conteo de Categorías

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     char tipos[] = {'A', 'B', 'A', 'C', 'A', 'B'};
5     int contA = 0;
6     for(int i = 0; i < 6; i++)
7         if(tipos[i] == 'A')
8             contA++;
9     printf("Categoria A: %d veces\n", contA);
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int contarCategoria(char tipos[], int n, char categoria) {
4     int cont = 0;
5     for(int i = 0; i < n; i++) {
6         if(tipos[i] == categoria)
7             cont++;
8     }
9     return cont;
10 }
11
12 int main() {
13     char tipos[] = {'A', 'B', 'A', 'C', 'A', 'B'};
14     int contA = contarCategoria(tipos, 6, 'A');
15     printf("Categoria A: %d veces\n", contA);
16     return 0;
17 }
```

Salida

Categoria A: 3 veces

Ejercicio 9: Búsqueda Lineal

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int ids[] = {101, 203, 305, 407, 509};
5     int buscar = 305, i = 0;
6     while(i < 5 && ids[i] != buscar)
7         i++;
8     if(i < 5)
9         printf("Encontrado en posicion %d\n", i);
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int buscarElemento(int arr[], int n, int valor) {
4     int i = 0;
5     while(i < n && arr[i] != valor)
6         i++;
7     if(i < n)
8         return i; // Posicion encontrada
9     return -1;    // No encontrado
10 }
11
12 int main() {
13     int ids[] = {101, 203, 305, 407, 509};
14     int buscar = 305;
15     int pos = buscarElemento(ids, 5, buscar);
16
17     if(pos != -1)
18         printf("Encontrado en posicion %d\n", pos);
19     else
20         printf("No encontrado\n");
21
22     return 0;
23 }
```

Salida

Encontrado en posicion 2

Ejercicio 10: Método de Newton-Raphson

Código Original

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main() {
5     float x = 10, tol = 0.01, x_nuevo;
6     while(fabs(x - 5) > tol) {
7         x_nuevo = (x + 5/x) / 2;
8         x = x_nuevo;
9     }
10    printf("Raiz: %.3f\n", x);
11    return 0;
12 }
```

Código con Funciones

```
1 #include <stdio.h>
2 #include <math.h>
3
4 float calcularRaiz(float objetivo, float x_inicial, float tol) {
5     float x = x_inicial;
6     float x_nuevo;
7
8     while(fabs(x - objetivo) > tol) {
9         x_nuevo = (x + objetivo/x) / 2;
10        x = x_nuevo;
11    }
12    return x;
13 }
14
15 int main() {
16     float x = 10, tol = 0.01;
17     float raiz = calcularRaiz(5, x, tol);
18     printf("Raiz: %.3f\n", raiz);
19     return 0;
20 }
```

Salida

Raiz: 2.236

Ejercicio 11: Codificación por Nivel

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     char nivel = 'B';
5     switch(nivel) {
6         case 'A': printf("Codigo: 1\n"); break;
7         case 'B': printf("Codigo: 2\n"); break;
8         case 'C': printf("Codigo: 3\n"); break;
9     }
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int obtenerCodigo(char nivel) {
4     switch(nivel) {
5         case 'A': return 1;
6         case 'B': return 2;
7         case 'C': return 3;
8         default: return 0;
9     }
10 }
11
12 int main() {
13     char nivel = 'B';
14     int codigo = obtenerCodigo(nivel);
15     printf("Codigo: %d\n", codigo);
16     return 0;
17 }
```

Salida

Codigo: 2

Ejercicio 12: Selección de Modelo

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int modelo = 2;
5     switch(modelo) {
6         case 1: printf("Regresion Lineal\n"); break;
7         case 2: printf("Random Forest\n"); break;
8         case 3: printf("SVM\n"); break;
9     }
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void seleccionarModelo(int modelo) {
4     switch(modelo) {
5         case 1:
6             printf("Regresion Lineal\n");
7             break;
8         case 2:
9             printf("Random Forest\n");
10            break;
11         case 3:
12             printf("SVM\n");
13             break;
14         default:
15             printf("Modelo desconocido\n");
16     }
17 }
18
19 int main() {
20     int modelo = 2;
21     seleccionarModelo(modelo);
22     return 0;
23 }
```

Salida

Random Forest

Ejercicio 13: Filtrado de Precios

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float precios[] = {45.5, 67.8, 34.2, 89.1, 23.7};
5     for(int i = 0; i < 5; i++)
6         if(precios[i] > 50)
7             printf("%.1f ", precios[i]);
8     return 0;
9 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void filtrarPrecios(float precios[], int n, float limite) {
4     for(int i = 0; i < n; i++) {
5         if(precios[i] > limite) {
6             printf("%.1f ", precios[i]);
7         }
8     }
9     printf("\n");
10 }
11
12 int main() {
13     float precios[] = {45.5, 67.8, 34.2, 89.1, 23.7};
14     filtrarPrecios(precios, 5, 50);
15     return 0;
16 }
```

Salida

67.8 89.1

Ejercicio 14: Comparación de Modelos

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float rmse1 = 2.5, rmse2 = 3.1;
5     printf("Mejor modelo: %s\n",
6           rmse1 < rmse2 ? "Modelo 1" : "Modelo 2");
7     return 0;
8 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int compararModelos(float rmse1, float rmse2) {
4     // Retorna 1 si modelo1 es mejor, 2 si modelo2 es mejor
5     return (rmse1 < rmse2) ? 1 : 2;
6 }
7
8 int main() {
9     float rmse1 = 2.5, rmse2 = 3.1;
10    int mejor = compararModelos(rmse1, rmse2);
11    printf("Mejor modelo: Modelo %d\n", mejor);
12    return 0;
13 }
```

Salida

Mejor modelo: Modelo 1

Ejercicio 15: Elegibilidad de Crédito

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int edad = 25;
5     float ingreso = 35000;
6     if(edad >= 18 && edad <= 65 && ingreso > 30000)
7         printf("Elegible para credito\n");
8     return 0;
9 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int esElegibleCredito(int edad, float ingreso) {
4     if(edad >= 18 && edad <= 65 && ingreso > 30000)
5         return 1; // Elegible
6     return 0;     // No elegible
7 }
8
9 int main() {
10     int edad = 25;
11     float ingreso = 35000;
12
13     if(esElegibleCredito(edad, ingreso))
14         printf("Elegible para credito\n");
15     else
16         printf("No elegible para credito\n");
17
18     return 0;
19 }
```

Salida

Elegible para credito

Ejercicio 16: Validación de Condiciones

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float ph = 7.2, temp = 22;
5     if((ph >= 6.5 && ph <= 7.5) || temp < 25)
6         printf("Condiciones aceptables\n");
7     return 0;
8 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 int condicionesAceptables(float ph, float temp) {
4     if((ph >= 6.5 && ph <= 7.5) || temp < 25)
5         return 1; // Aceptable
6     return 0;     // No aceptable
7 }
8
9 int main() {
10     float ph = 7.2, temp = 22;
11
12     if(condicionesAceptables(ph, temp))
13         printf("Condiciones aceptables\n");
14     else
15         printf("Condiciones no aceptables\n");
16
17     return 0;
18 }
```

Salida

Condiciones aceptables

Ejercicio 17: Cálculo de Accuracy

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     int fallos = 0, precisos = 0;
5     int pred[] = {1, 0, 1, 1, 0};
6     int real[] = {1, 0, 0, 1, 0};
7     for(int i = 0; i < 5; i++)
8         pred[i] == real[i] ? precisos++ : fallos++;
9     printf("Accuracy: %.1f%%\n", (precisos*100.0)/5);
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 float calcularAccuracy(int pred[], int real[], int n) {
4     int precisos = 0;
5     for(int i = 0; i < n; i++) {
6         if(pred[i] == real[i])
7             precisos++;
8     }
9     return (precisos * 100.0) / n;
10 }
11
12 int main() {
13     int pred[] = {1, 0, 1, 1, 0};
14     int real[] = {1, 0, 0, 1, 0};
15     float accuracy = calcularAccuracy(pred, real, 5);
16     printf("Accuracy: %.1f%%\n", accuracy);
17     return 0;
18 }
```

Salida

Accuracy: 80.0%

Ejercicio 18: Generación de Muestra Aleatoria

Código Original

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int muestra[5], i = 0;
6     do {
7         muestra[i] = rand() % 100;
8         printf("%d ", muestra[i]);
9     } while(++i < 5);
10    return 0;
11 }
```

Código con Funciones

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void generarMuestra(int muestra[], int n, int max) {
5     for(int i = 0; i < n; i++) {
6         muestra[i] = rand() % max;
7     }
8 }
9
10 void imprimirArreglo(int arr[], int n) {
11     for(int i = 0; i < n; i++) {
12         printf("%d ", arr[i]);
13     }
14     printf("\n");
15 }
16
17 int main() {
18     int muestra[5];
19     generarMuestra(muestra, 5, 100);
20     imprimirArreglo(muestra, 5);
21     return 0;
22 }
```

Salida

83 86 77 15 93

Nota: Los valores varían en cada ejecución debido a la naturaleza aleatoria.

Ejercicio 19: Clasificación de Scores

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float scores[] = {0.72, 0.45, 0.89, 0.33};
5     for(int i = 0; i < 4; i++)
6         printf("%s ", scores[i] >= 0.5 ? "Positivo" : "Negativo");
7     return 0;
8 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void clasificarScore(float score) {
4     if(score >= 0.5)
5         printf("Positivo ");
6     else
7         printf("Negativo ");
8 }
9
10 void clasificarScores(float scores[], int n) {
11     for(int i = 0; i < n; i++) {
12         clasificarScore(scores[i]);
13     }
14     printf("\n");
15 }
16
17 int main() {
18     float scores[] = {0.72, 0.45, 0.89, 0.33};
19     clasificarScores(scores, 4);
20     return 0;
21 }
```

Salida

Positivo Negativo Positivo Negativo

Ejercicio 20: Imputación de Valores Faltantes

Código Original

```
1 #include <stdio.h>
2
3 int main() {
4     float datos[] = {23.5, -999, 34.2, -999, 28.7};
5     float media = 28.8;
6     for(int i = 0; i < 5; i++)
7         printf("%.1f ", datos[i] == -999 ? media : datos[i]);
8     return 0;
9 }
```

Código con Funciones

```
1 #include <stdio.h>
2
3 void imputarValoresFaltantes(float datos[], int n,
4                             float valor_faltante, float imputar) {
5     for(int i = 0; i < n; i++) {
6         if(datos[i] == valor_faltante) {
7             datos[i] = imputar;
8         }
9     }
10 }
11
12 void imprimirDatos(float datos[], int n) {
13     for(int i = 0; i < n; i++) {
14         printf("%.1f ", datos[i]);
15     }
16     printf("\n");
17 }
18
19 int main() {
20     float datos[] = {23.5, -999, 34.2, -999, 28.7};
21     float media = 28.8;
22
23     imputarValoresFaltantes(datos, 5, -999, media);
24     imprimirDatos(datos, 5);
25
26     return 0;
27 }
```

Salida

23.5 28.8 34.2 28.8 28.7